Atal Bihari Vajpayee
Indian Institute of Information Technology
and Management, Gwalior



MeterTrace

Smart-Meter Data Verification and Review

Project report
IoT Big Data Management

Submitted by

Nishant Harkut
Enrollment number: 2023IMG-040

Submitted to

Dr. Amandeep Kaur
Instructor

13 November 2026

Project repository

`https://github.com/nishantharkut-research/IoT-Energy-Data-Pipeline`

Abstract

Smart-meter data can reach an analytics platform through repeated, delayed or malformed deliveries. A successful processing job alone gives little information about whether these deliveries produced a complete, usable result. This project builds an auditable energy-data pipeline that preserves received deliveries, reconstructs exact source records and checks the finalized result through a second implementation.

Public HKUST smart-meter workbooks and Brick metadata are registered and converted into exact canonical records. Deterministic Kafka replay introduces controlled delivery faults. Spark stores deliveries in Bronze, maintains an event-time-bounded Silver view and rebuilds finalized Gold from Bronze. Independent Hadoop Streaming programs compute reference records and aggregates from canonical NDJSON. Delivery accounting, replay termination and exact comparisons must agree before publication.

Fifteen runs were completed: nine correctness, recovery and watermark fixtures, plus three repetitions each at 100K and 1M source records. All declared Spark/Hadoop comparisons agree. A restrictive watermark leaves 143 records in Silver, while Gold retains all 144 source records. Interrupted and uninterrupted fixture executions reproduce four semantic outputs. Median verification time is 13.88 minutes at 100K and 165.30 minutes at 1M.

A read-only dashboard presents source quality, delivery dispositions, verification, recovery and storage/query costs. Consumption analysis distinguishes matching intervals from unsupported comparisons: the measured discrepancy is zero on 138 comparable periods, while two unmatched periods leave total exposure unknown. The proposed Indian application is a DISCOM review of metering-provider data, evaluated here using historical HKUST replay rather than a live utility deployment.

Keywords: smart-meter data, delivery accounting, Apache Kafka, Apache Spark, Hadoop Streaming, independent verification.

Contents

1 Introduction **4**

2 System architecture and technology choices **4**

3 Dataset and source preparation **6**

4 Delivery processing and finalization **8**

5 Independent Hadoop verification **9**

6 Experiment design and controlled-fault results **10**

7 Watermark behaviour and consumption interpretation **11**

8 Recovery and reproducibility of the finalized answer **13**

9 Scale, storage and SQL measurements **13**

10 Dashboard implementation and utility-review workflow **15**

11 Discussion and conclusion **16**

Reading the report. Sections 1–3 explain the problem, architecture and data. Sections 4–5 describe processing and verification. Sections 6–9 present measured results. Section 10 explains the dashboard and proposed utility-review workflow. Figures are placed with their discussion. Full desktop captures are also supplied with the project.

Dashboard views in this report

View	Page
Overview and selected-run accounting	6
Source audit	7
Delivery inspector	9
Fault comparison	11
Independent verification	10
Watermark and consumption review	12
Recovery	13
Storage and SQL	14
Indian utility-review workflow	15
Meter display and review	16

1 Introduction

1.1 Problem and motivation

Smart-meter analytics must establish which readings arrived, were rejected, were repeated and contributed to the final result. Retries create extra deliveries, late records can miss the operational cutoff, and malformed messages require traceable exclusion. The project asks: *what evidence supports a finalized result when delivery contains such faults?* It preserves delivery history, reconstructs source records after completion and compares them with a second implementation.

1.2 Connection to Indian smart-meter systems

The intended user is a distribution-company (DISCOM) analyst reviewing an advanced metering infrastructure service provider's (AMISP) outputs. Head-end systems (HES) collect readings; meter-data management (MDM) applies validation, estimation and editing (VEE). Review must distinguish source gaps, delivery exceptions and processing differences.

A CSIS survey published on 30 October 2026 reports communication and data-integration limitations in TNPDCL and Tata Power-operated Odisha utilities [10]. A CAG report also records historical communication and reporting problems in a Tripura smart-grid project during 2018–2021 [11]. These are specific examples motivating data review, not estimates of national failure prevalence.

Raw-data retention, VEE and quality monitoring already exist. MSEDCL's 2022 tender requires original received data, VEE data and edit trails [7], while commercial MDM systems expose quality and timeliness metrics [8]. Quality control for AMI data is also established in the literature [9]. This project contributes an evaluated, reproducible integration of delivery accounting, exact reconstruction, independent computation and explicit comparison coverage. It is intended to support review alongside existing systems.

1.3 Objectives and completed scope

The implementation has five objectives:

1. Preserve and account for each received delivery, including retries and rejected messages.
2. Separate the bounded operational result from the exact finalized source result.
3. Verify source records and aggregates through independent Hadoop programs.
4. Demonstrate recovery without changing the declared semantic outputs.
5. Present quality, consumption support and measured processing costs in a usable review interface.

The evaluation contains 15 completed runs through 1M selected source records. The 5M experiments were excluded because of resource and time constraints. The Indian application is a proposed workflow, supported by public-source motivation and tested mechanisms. Field billing savings and live utility reliability are outside the evaluation.

2 System architecture and technology choices

Figure 1 shows the two computation paths. Source registration and canonicalization are shared. Kafka and Spark form the delivery path. Canonical NDJSON also enters HDFS for Hadoop Streaming. Both outputs are checked before finalization. The comparison is independent in implementation after canonicalization, not independent in physical measurement or source preparation.

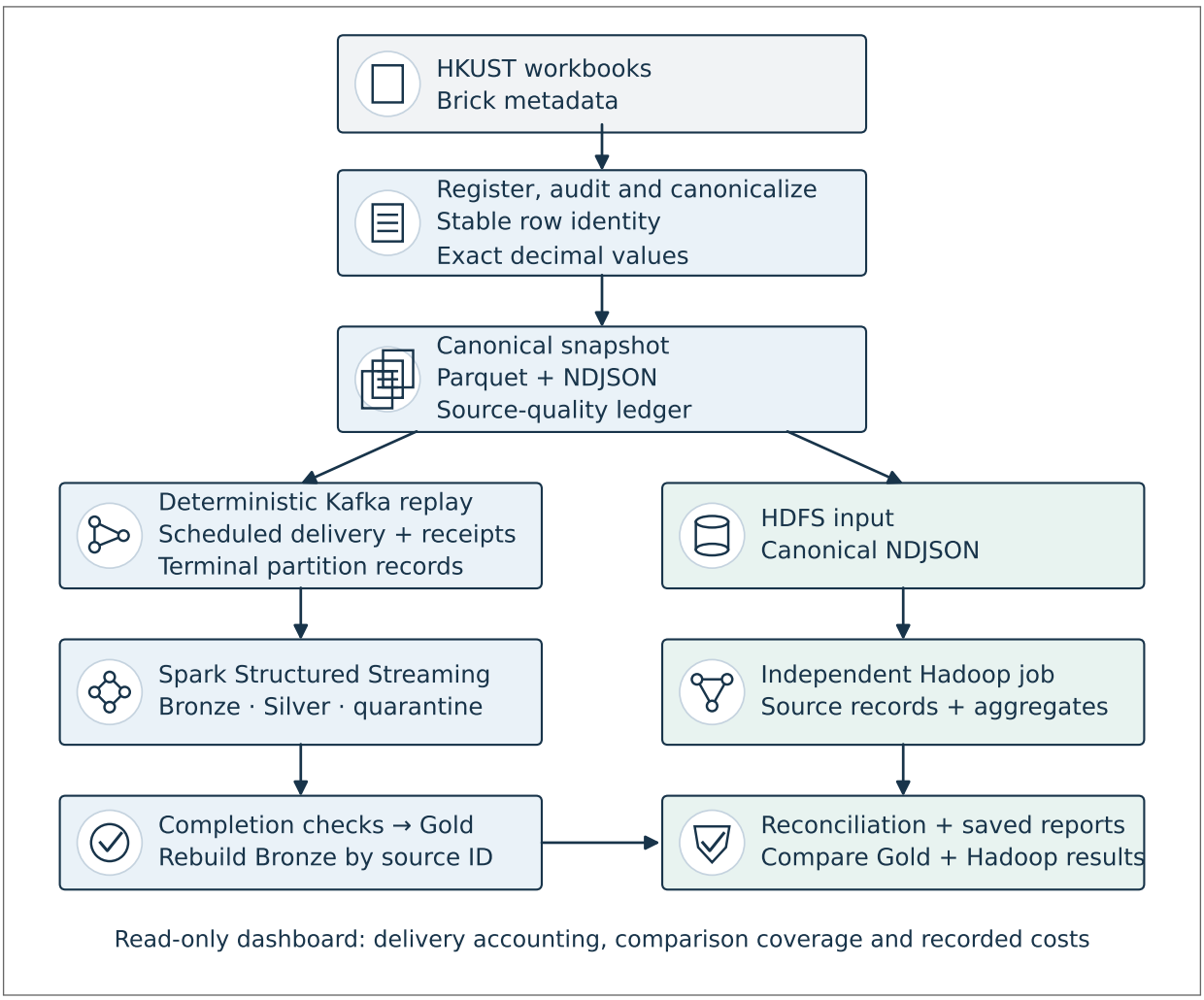


Figure 1: Processing path and independent reference computation share a declared canonical input.

Table 1: Technology choices and their role in the implementation.

Component	Role and reason for use
Kafka 3.7.1	Partitioned delivery log, acknowledgement tracking and reproducible replay [5].
Spark 3.5.1	Stateful event-time processing and batch reconstruction from retained deliveries [4].
Hadoop 3.4.3	HDFS input and standalone Streaming programs for a separately implemented reference [6].
NDJSON / Parquet	Explicit record interchange for verification / compressed columnar storage for analysis.
SQLite	Disk-backed ordering and interval processing with bounded working memory.
React / TypeScript	Read-only desktop review of saved results, with Mantine components and Recharts plots.
JSON manifests	Bind source, configuration, completed outputs and presentation inputs by recorded hashes.

2.1 Review workflow and storage decisions

The reviewer selects a completed experiment, checks source coverage, delivery accounting and exact comparisons, then interprets consumption and cost. Figure 2 shows the entry point; its experiment register links the remaining views to the selected run.

Files retain experiment outputs, SQLite supports local ordering, and Parquet/Spark SQL supply analytical queries. Historical replay and read-only review require no separate serving database such as HBase.

Docker Compose provides the experiment services. The React/TypeScript viewer uses Mantine components and Recharts plots. Its Python exporter preserves saved summary bytes, which the browser checks against SHA-256 bindings. A separate Streamlit viewer checks original sealed artifacts when available.

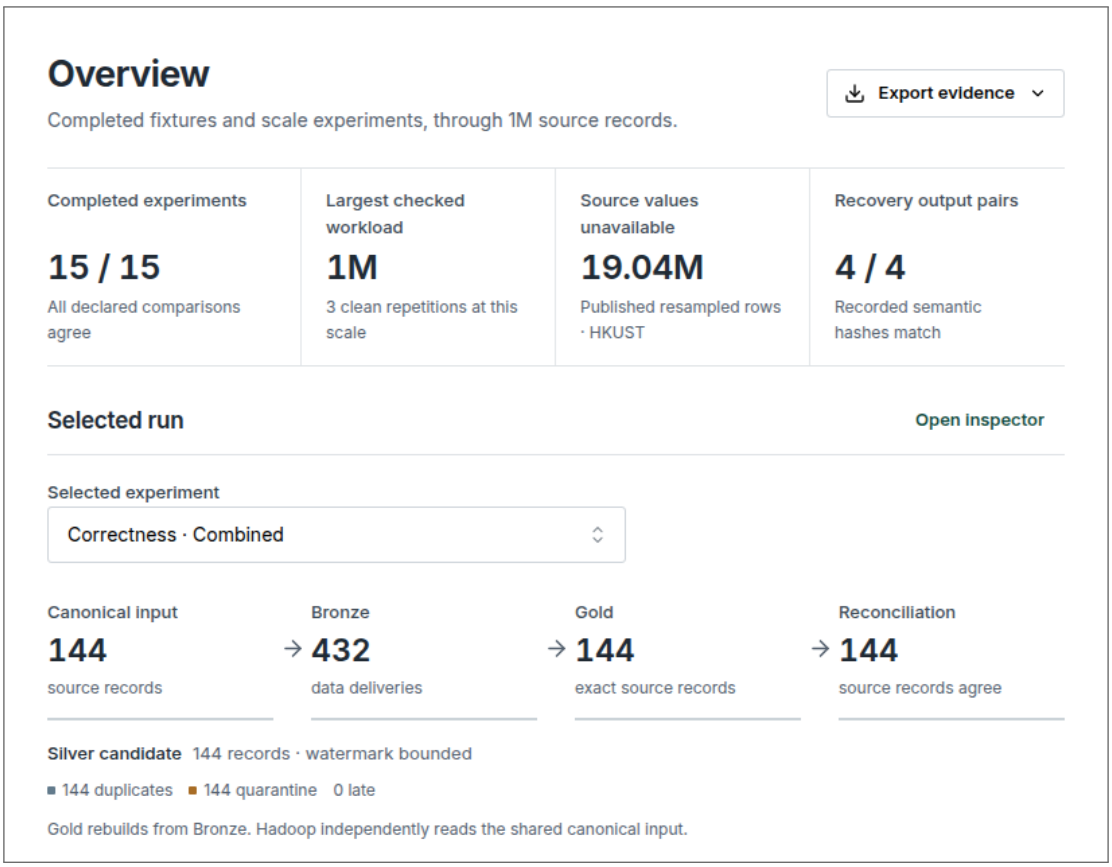


Figure 2: Completed-run overview and selected delivery path in MeterTrace.

3 Dataset and source preparation

3.1 Public telemetry and metadata

The source is version 5 of the HKUST campus smart-meter collection on Dryad, DOI `10.5061/dryad.k3j9kd5h6` [1, 2]. Raw and publisher-resampled variants remain separate. Brick metadata supplies semantic relationships and recorded units [3]; all workbook meters join to it.

The audit covers 1,394 raw and 1,329 resampled workbooks (Table 2). The 65 raw workbooks absent from the resampled set contain no physical data rows. Brick contains 1,432 meter references; that count alone does not establish missing telemetry for a particular window.

Table 2: Published source-audit counts. Categories can overlap.

Observation	Raw	Resampled
Workbooks	1,394	1,329
Physical data rows	50,903,710	63,273,828
Finite numeric readings	50,903,710	44,238,638
Missing numeric cells	0	19,035,190
Negative readings	134,612	134,612
Ordered numeric decreases	50,675	67,878
Duplicate decoded timestamps	853,595	0
Out-of-order timestamps	147	0

3.2 What availability means

Numeric-cell availability is 44,238,638/63,273,828 = 69.916%, using published rows as the denominator. This is not independently expected temporal coverage. Negative values and decreases are review observations rather than confirmed failures or resets. Figure 3 presents the source populations.

3.3 Registration and preservation

Registration records paths, sizes and SHA-256 hashes; profiling checks workbook structure, values and timestamp order. Preparation accounts for 63,273,828 rows as 44,238,638 numeric records and 19,035,190 source-quality records. Numeric records can retain quality flags. Source-quality exclusions concern preparation, while Kafka quarantine concerns delivered messages.

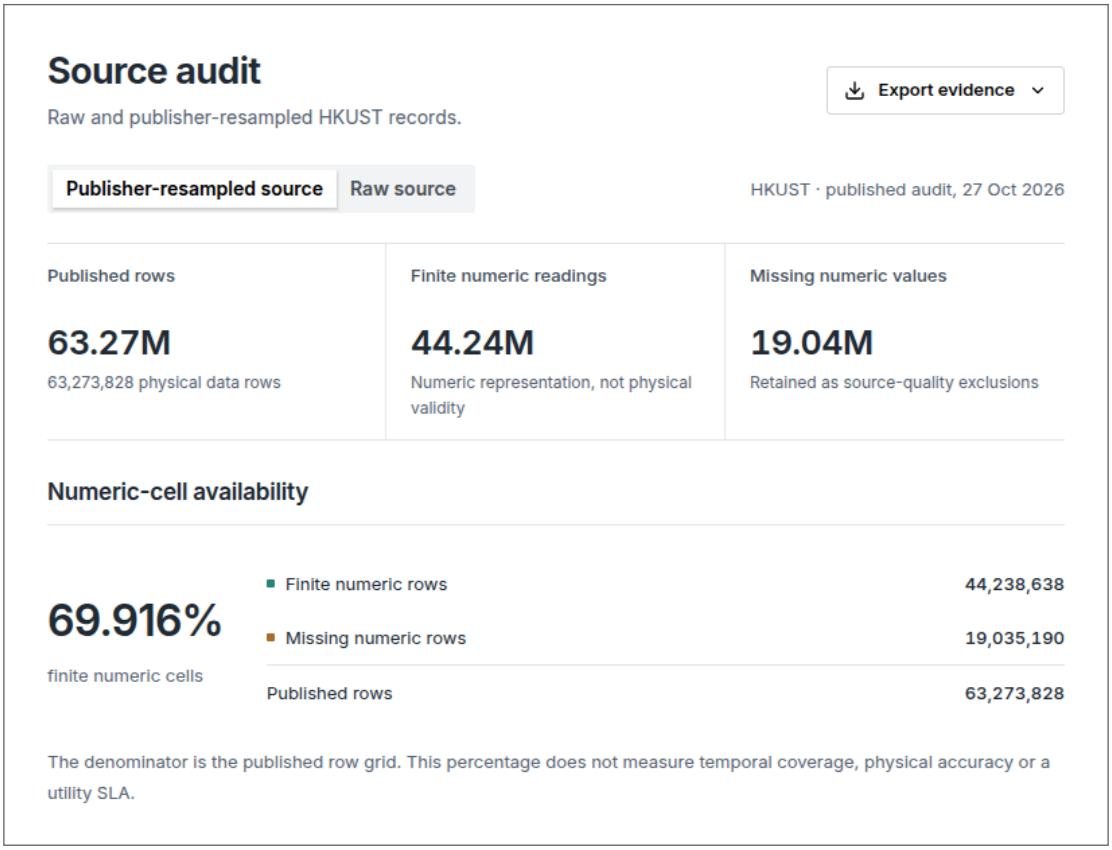


Figure 3: Raw/resampled selection, numeric-cell availability and source-registration counts.

3.4 Canonical identity and exact representation

3.4.1 Source rows and delivery identities

The source identifier is a SHA-256 digest of a structural locator:

$$\text{source_event_id} = H(v \parallel s \parallel p \parallel w \parallel r), \tag{1}$$

where v is dataset version, s variant, p relative path, w worksheet and r physical row. Serialization uses a declared separator and rejects ambiguous components. Repeated deliveries share source identity but retain distinct delivery identities.

Distinct physical rows remain distinct even when timestamps match. Canonical records retain the locator, original timestamp/numeric text, interpreted event time, scaled value, decimal scale, unit, measurement kind, Brick relationships and quality flags. Gold deduplicates source identities; consumption derivation applies its timestamp policy afterwards.

3.4.2 Numeric precision and time interpretation

For decimal x at scale d, the exact stored integer is $z = x10^d$. Original cell text is retained before binary floating-point conversion. Meter D0779's `26724.7` at scale 15 requires `26724700000000000000`, exceeding signed 64-bit range. The contract therefore checks a signed 38-digit decimal envelope through the Spark representation path.

The audited timestamps are naive. The snapshot interprets them in `Asia/Hong_Kong`, an experimental assumption unconfirmed by the authors. Original timestamp text is retained; UTC boundaries depend on that assumption.

3.4.3 Bounded-memory preparation

Preparation checkpoints sorted workbook units and merges bounded groups into canonical NDJSON, the quality ledger, Parquet and snapshot manifests. The full source is prepared once. Scale workloads select source-identifier-sorted numeric prefixes, which are not continuous meter-days suitable for outage-duration or time-of-use analysis.

4 Delivery processing and finalization

4.1 Replay records and terminal evidence

A deterministic schedule fixes source selections, seed, partitions and faults. Receipts retain delivery/source identities and acknowledgements. Terminal records establish completion for each partition. Bronze archives deliveries and transport evidence; terminal controls are checked separately. Parsing sends invalid payloads to categorized quarantine, preserving their exclusion reasons.

4.2 Operational Silver and finalized Gold

Silver uses event-time watermarking to bound state and duplicate suppression [4]; late-valid records are distinct from invalid messages. After completion checks, batch Gold rebuilds valid source records and aggregates from Bronze, deduplicating exact source identities. It retains received valid records omitted by Silver's watermark, while genuinely unreceived readings remain unavailable. Figure 4 shows both paths.

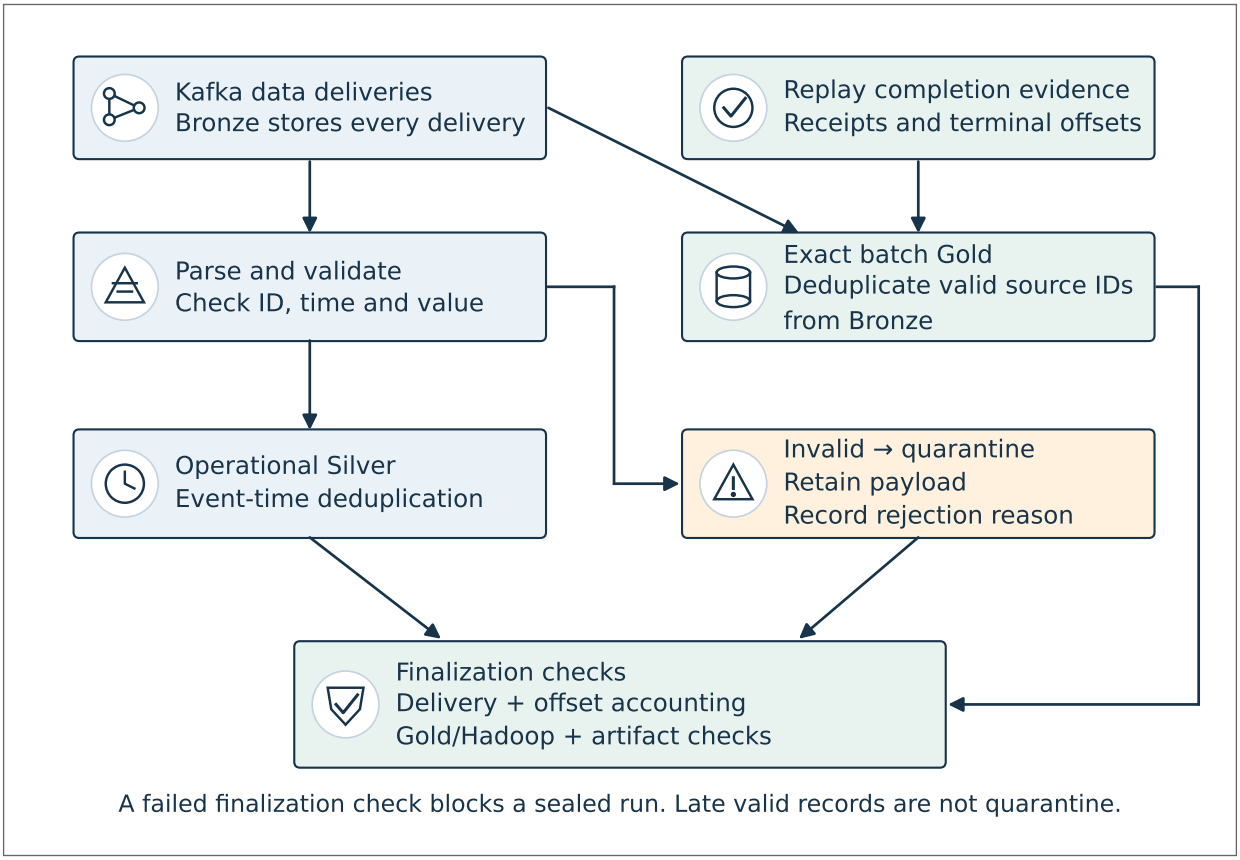


Figure 4: Delivery dispositions and exact finalization. Gold uses Bronze rather than inheriting Silver's omissions.

Figure 5 presents the selected run and its delivery accounting. The layer audit checks two conservation equations. Let B be Bronze data deliveries, V parsed-valid deliveries, Q quarantine deliveries, S Silver records, D duplicate-valid dispositions and L late-valid dispositions. Under the recorded run definitions,

$$B = V + Q, \tag{2}$$

$$V = S + D + L. \tag{3}$$

The completion check separately binds acknowledged data deliveries to Bronze and verifies terminal partition coverage. The exact Gold count is a source-event count, not another delivery-disposition category.

4.3 Fail-closed publication of completed runs

Finalization requires the replay manifests, delivery totals, terminal offsets, layer dispositions and exact comparisons to reconcile. Decision, query and storage reports must bind to the corresponding finalized artifacts. A run manifest registers the artifacts and is sealed with a SHA-256 binding. The original sealed-run dashboard checks both this manifest and its registered files and rejects incomplete, altered or unreconciled runs. These checks establish byte bindings and declared processing consistency, not adversarial authenticity or physical accuracy.

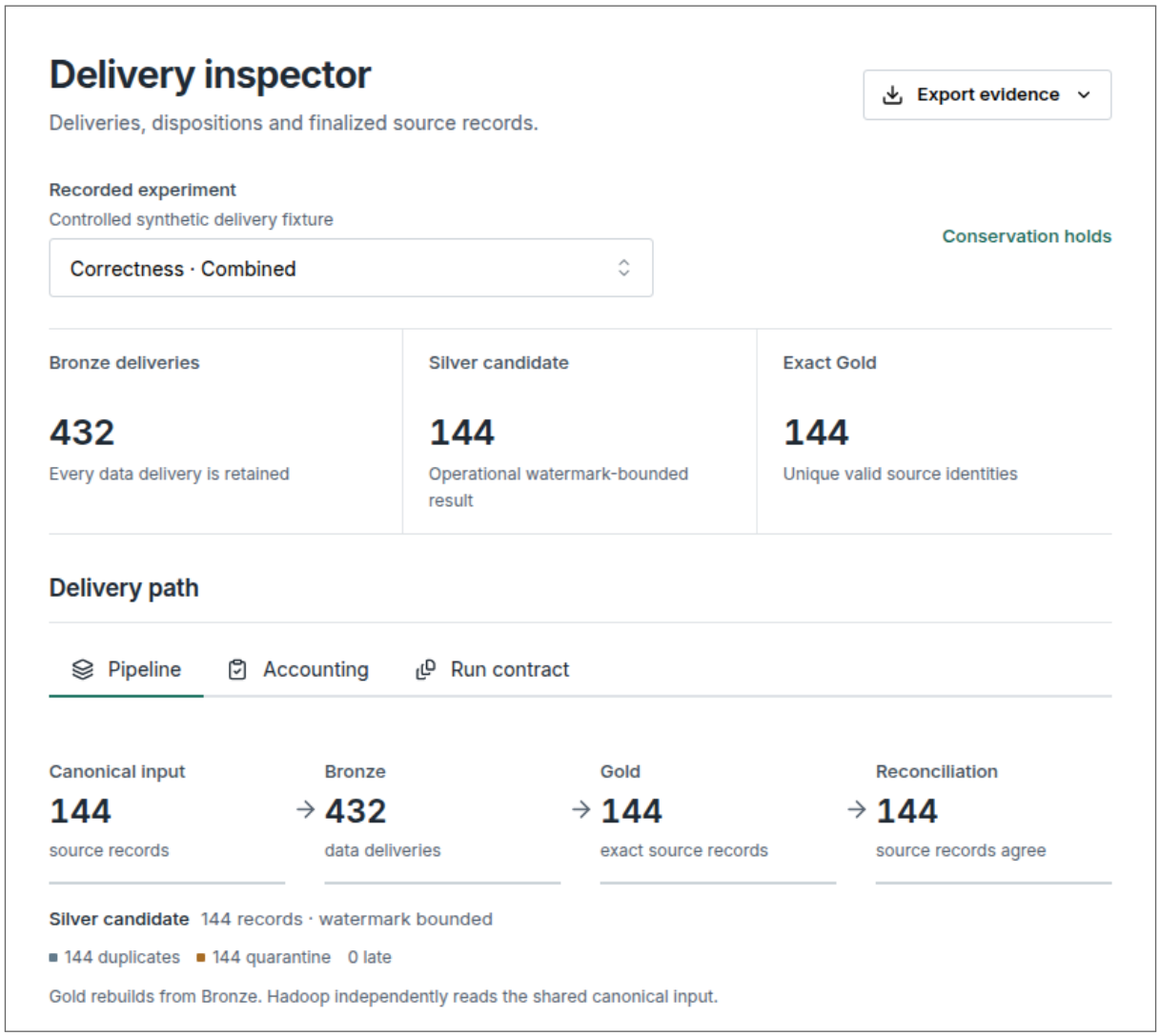


Figure 5: Selected-run delivery accounting and processing layers.

5 Independent Hadoop verification

Standalone Hadoop programs read canonical NDJSON independently of Spark transformations. They emit source records and grouped counts, exact scaled-value sums, timestamp bounds and quality counts. Grouping includes meter, unit, measurement kind and decimal scale; Python integers preserve exact sums.

Sorted reconciliation compares 23 source fields and 10 aggregate fields (Figure 6). Source fields include location, original text, event time, scale, value, units, quality flags and Brick relationships. All 15 runs agree through 1M selected source records. Aggregate sums of cumulative registers are processing checksums; consumption requires successive-reading differences. Shared canonical preparation remains a common dependency, so agreement establishes processing consistency rather than physical truth.

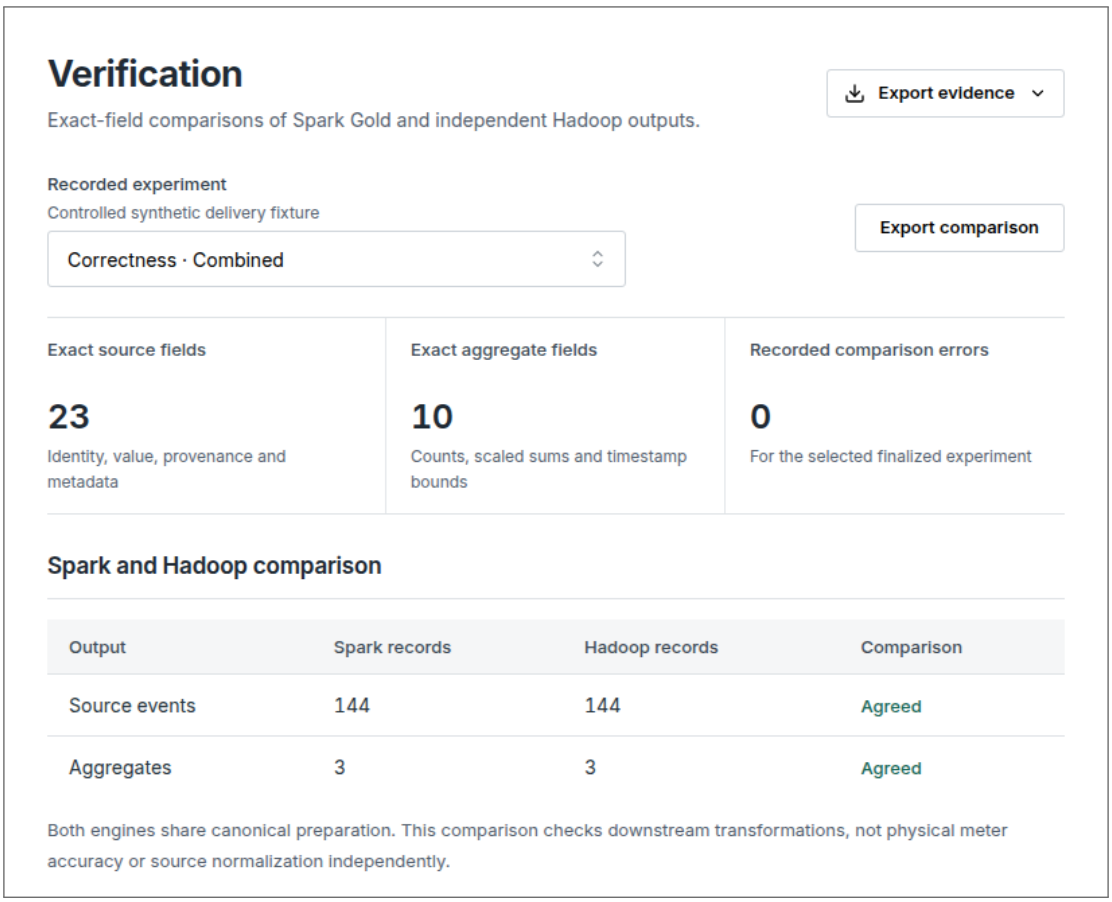


Figure 6: Recorded Spark/Hadoop source and aggregate comparisons with the exact-field contract.

6 Experiment design and controlled-fault results

The final matrix contains five correctness schedules, two recovery executions, two watermark configurations and six real-data scale executions. Each correctness schedule uses a 144-source-event fixture. The clean scale workloads use 100K or 1M selected canonical numeric records, with three repetitions at each scale. The originally planned 5M repetitions are explicitly excluded for resource and time constraints.

Table 3: Executed streaming configurations. All use the declared stack in Table 1.

Family	**Partitions**	**Offsets/trigger**	**Watermark**
Correctness / recovery	3	60	24 hours
Watermark comparison	1	1	24 hours / 1 minute
Clean scale	3	10,000	24 hours

Table 4: Fixture layer counts from the saved closure report. Terminal controls are separate.

Fixture	Bronze	Silver	Dup.	Late	Quar.	Gold
clean	144	144	0	0	0	144
duplicate	288	144	144	0	0	144
delayed	144	144	0	0	0	144
malformed	288	144	0	0	144	144
combined	432	144	144	0	144	144
recovery-uninterrupted	144	144	0	0	0	144
recovery-interrupted_resumed	144	144	0	0	0	144
watermark-retaining	144	144	0	0	0	144
watermark-restrictive	144	143	0	1	0	144

Figure 7 compares the five delivery schedules. The duplicate schedule repeats unchanged valid deliveries. The malformed schedule adds rejected messages alongside valid originals. The combined schedule includes both effects. Consequently, these schedules test delivery preservation and disposition under declared faults. They do not simulate arbitrary physical-meter corruption or guarantee a nonzero consumption error.

In the combined fixture, 432 Bronze deliveries split into 288 parsed-valid and 144 quarantined deliveries. The valid deliveries split into 144 Silver records and 144 duplicate dispositions. Gold contains the same 144 source records as the clean execution. The five correctness schedules have identical hashes for each of the four corresponding semantic outputs: Gold source records, Gold aggregates, Hadoop source records and Hadoop aggregates. This is stronger than merely reporting equal row totals.

All 15 completed runs have empty source/aggregate error lists. The controlled schedules evaluate processing behaviour, rather than field fault prevalence.

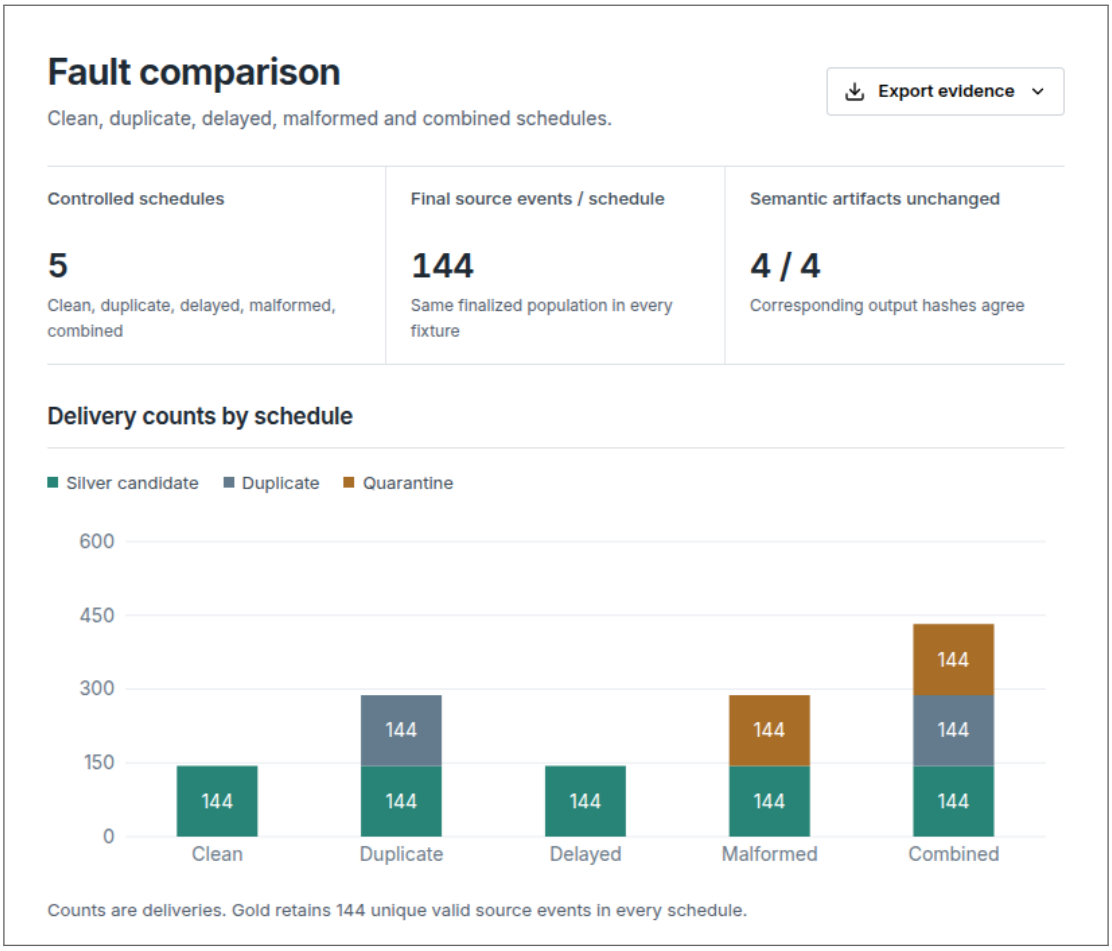


Figure 7: Delivery dispositions across the five controlled schedules.

7 Watermark behaviour and consumption interpretation

The retaining watermark fixture preserves 144 source records in Silver. The restrictive configuration preserves 143, with the remaining valid record classified as late. Both executions produce identical finalized Gold and Hadoop source records. Thus the observed difference is an operational candidate omission, not a failure of exact Gold reconstruction.

Consumption is derived only for records whose declared semantics support cumulative kWh. For two eligible readings, interval energy is

$$\Delta E_{m,i} = E_m(t_i) - E_m(t_{i-1}). \tag{4}$$

The derivation requires a valid prior baseline and unambiguous timestamps. Negative differences and unresolved resets are flagged. The conservative duplicate-timestamp policy also excludes affected derivations and the first subsequent interval requiring a new baseline.

A numeric comparison requires the same meter, start time and end time in both series. Let C denote that comparable-support set. The recorded discrepancy is

$$D_C = \sum_{(m,a,b)\in C} \left| \Delta E^{\text{Silver}}_{m,a,b} - \Delta E^{\text{Gold}}_{m,a,b} \right|. \tag{5}$$

It is a subtotal over C, not an error estimate for intervals outside it.

Table 5: Consumption support in the restrictive-watermark execution.

Observation	Recorded result
Candidate / verified intervals	140 / 138
Comparable / incomparable periods	138 / 2
Candidate / verified flagged records	3 / 6
Absolute discrepancy on comparable support	0 kWh
Total residual exposure	Unknown

Table 5 and Figure 8 distinguish comparable support from the two unmatched periods, whose error is unknown. Hypothetical rates of 0.5, 1.0 and 2.0 currency units/kWh yield zero comparable-support scenario subtotals. They are not utility tariffs; total energy error and financial exposure remain unknown.

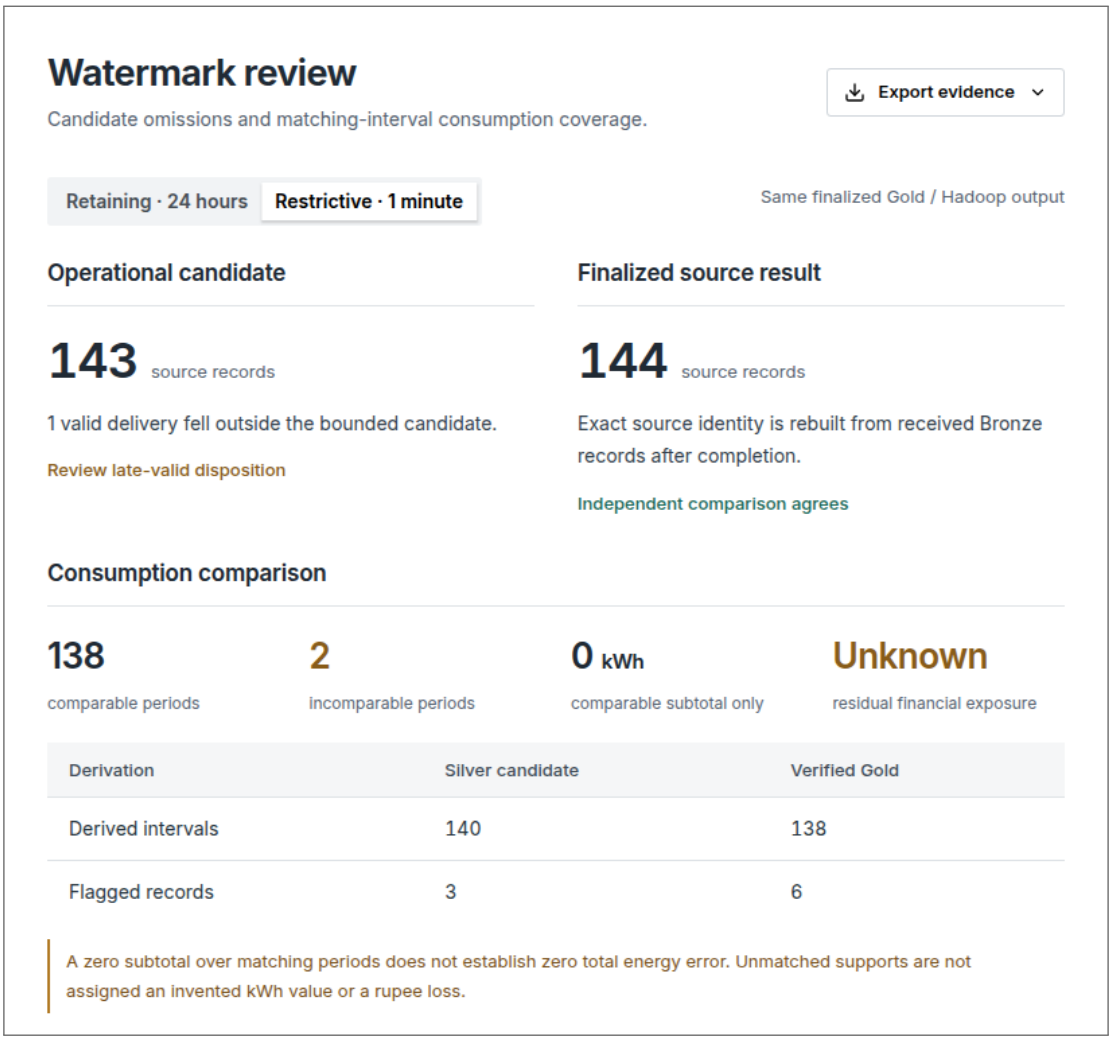


Figure 8: Restrictive Silver omission, exact Gold and consumption-comparison coverage.

8 Recovery and reproducibility of the finalized answer

The uninterrupted and resumed recovery fixtures have identical SHA-256 digests for all four semantic outputs (Figure 9). Transport offsets, timings and physical Parquet layouts describe execution details and need not match.

All three 1M executions also recovered and finalized. Receipts record resumed stages and configuration bindings. Component costs and exact comparisons are available, but complete workflow durations are unavailable.

The portable loader checks summary bytes, plan/scope bindings, 15 run identities and conservation equations. Original runtime verification dates to 2 November 2026; repeating the per-artifact audit requires the original sealed directories.

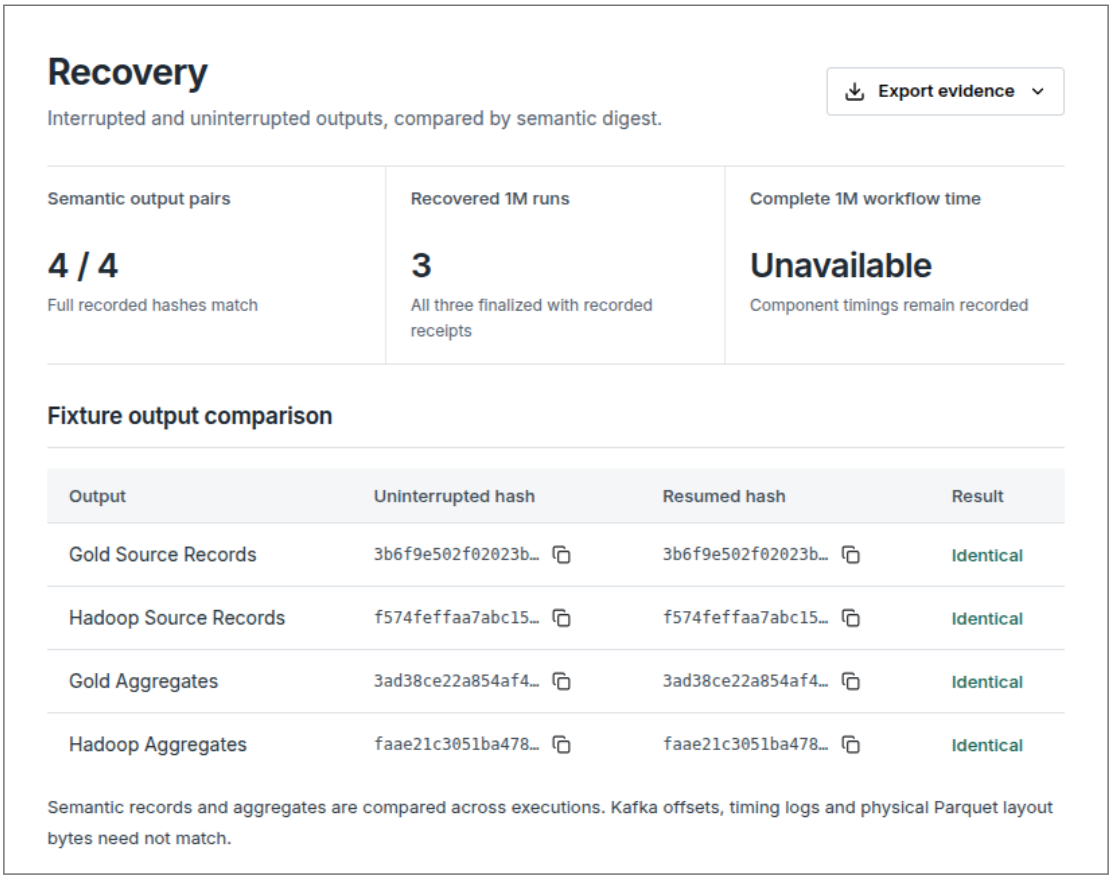


Figure 9: Matching fixture output digests and saved scale-recovery receipts.

9 Scale, storage and SQL measurements

9.1 Verification cost

Table 6 covers Gold, layer audit, Hadoop and reconciliation across three repetitions. Time excludes streaming and subsequent benchmarks; storage includes finalized verification artifacts, excluding checkpoints, logs and service volumes. The 1M median is 11.91 times the 100K median for ten times as many events. This describes verification

Table 6: Recorded verification costs. Time is minutes; storage is decimal MB.

Events	Minimum	Median	Maximum	Median storage
100,000	13.63	13.88	14.10	252.78
1,000,000	163.42	165.30	165.97	2522.71

cost on one host, not production throughput or asymptotic complexity. CPU/RAM specifications are absent from the portable closure; the plan binds the original machine metadata.

9.2 Storage formats

Figure 10 shows costs and storage. Table 7 reports stored bytes divided by NDJSON bytes; ratios repeat across all three executions per scale. Unpartitioned Zstandard Parquet is smaller, while partitioning adds file/metadata overhead. Query-speed benefits were not established.

Table 7: Recorded storage sizes and ratios for each scale.

Events	NDJSON (MB)	Parquet/NDJSON	Partitioned/NDJSON
100,000	112.04	0.0615	0.3038
1,000,000	1120.65	0.0601	0.1654

9.3 Query workloads and timing limits

Five queries cover meter, building, time window, quality and exposure. Each has three repetitions and a complete ordered-result digest. Timings include cached Gold access, JSON serialization, transfer and hashing. Smaller query partitions at 1M prevent a controlled cross-scale timing comparison. Quality-query results concern selected numeric prefixes, not full-source missingness. Per-run timing ranges and digests are committed.

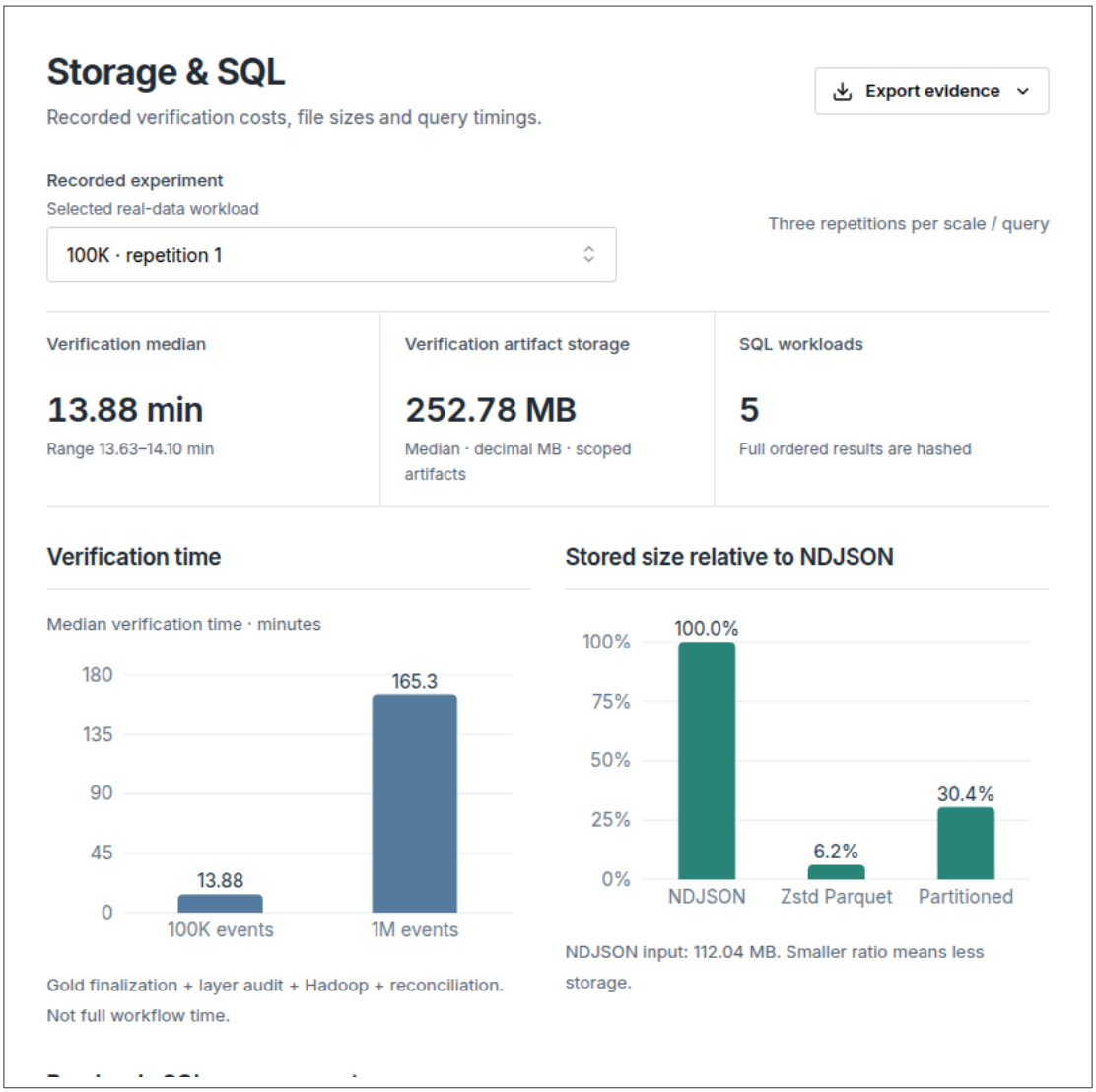


Figure 10: Verification cost and storage results for the selected repetition.

10 Dashboard implementation and utility-review workflow

10.1 From experiment results to a review interface

Ten views cover the overview, source audit, delivery inspector, faults, verification, watermark, recovery, storage/SQL, Indian use case and meter review. Run selection updates counts and comparison contracts. Source and watermark selectors expose the relevant populations and policies; search, run dialogs and CSV/JSON exports support inspection. A consistent light theme keeps units and review context visible.

The browser verifies exported summary hashes and rejects missing or altered inputs. Reviewing these saved results needs no experiment services or external SSD. The separate Streamlit viewer performs original artifact checks when sealed directories are available.

10.2 Proposed Indian utility application

Figure 11 maps the implemented mechanisms to DISCOM review alongside HES/MDM/VEE: identify source gaps, account for deliveries, check the finalized computation and interpret consumption coverage. Source gaps require follow-up, processing exceptions require disposition, and unmatched intervals retain unknown exposure.

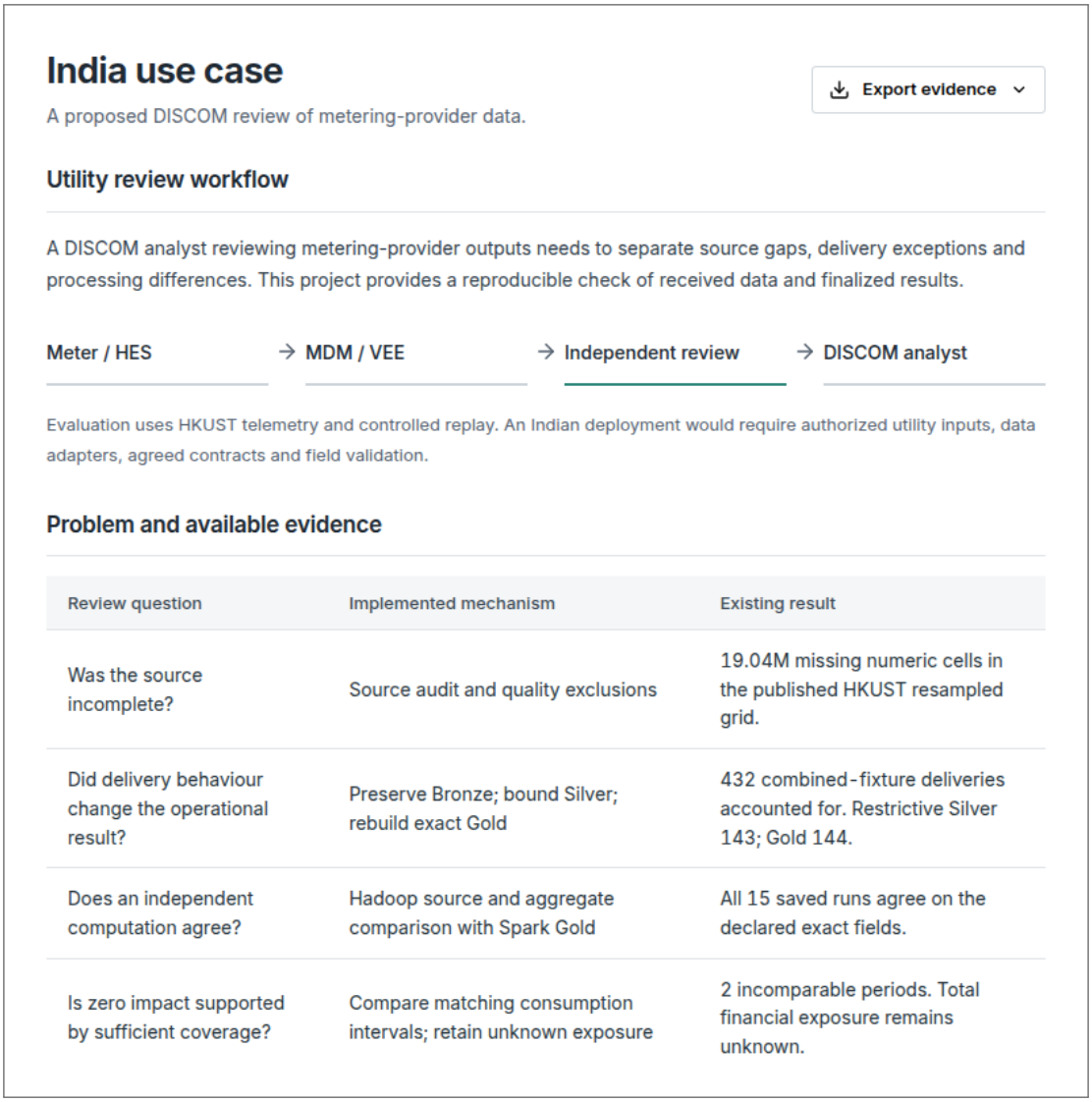


Figure 11: Proposed DISCOM review workflow and its connection to the implemented checks.

10.3 Interactive meter-review demonstration

The meter-review workspace (Figure 12) links a selectable kWh register to three configured cases: late reading, duplicate receipt and source gap. Reviewers compare readings, add notes, mark cases reviewed and export decisions as JSON; reset clears session state.

Demonstration readings of 1000, 1001, 1003 and 1004 kWh preserve a 4 kWh endpoint difference. Late recovery restores interval boundaries; deduplication retains one record per source event. A source gap leaves half-hour allocations unresolved and prompts follow-up. Experimental evidence remains the 15 measured runs.

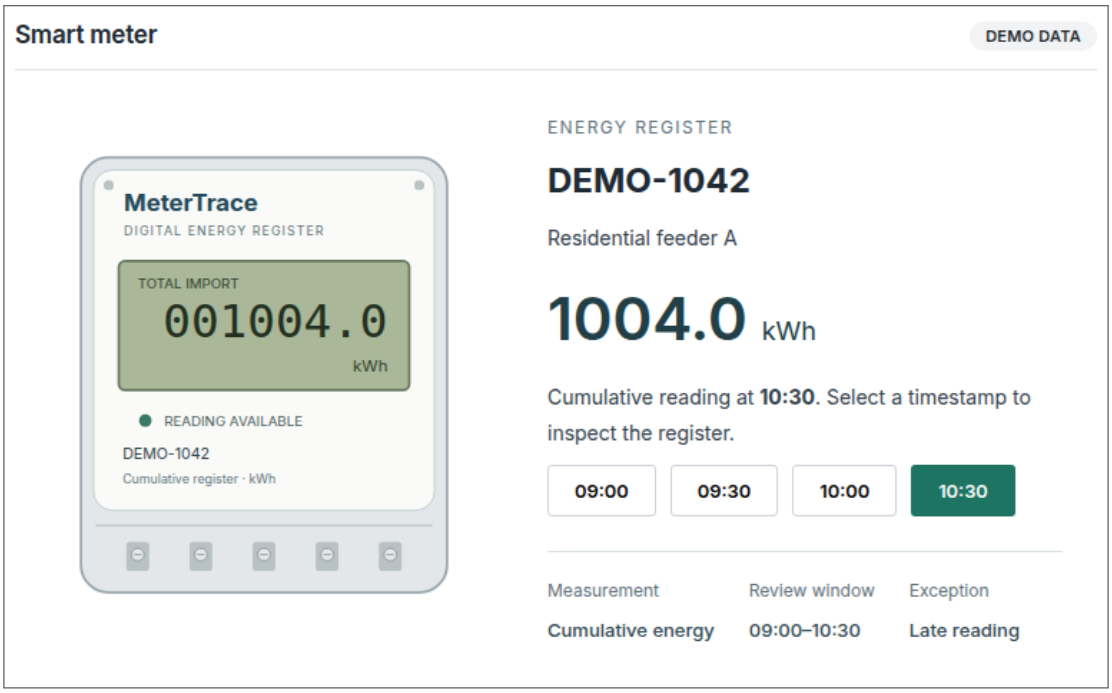


Figure 12: Digital meter display with cumulative kWh register and timestamp selection.

10.4 Software verification and handover

The submission checks cover parsing and precision, deterministic identity, replay contracts, conservation, finalization, independent computation and presentation bindings. The current non-distributed verification records 582 passing Python tests and seven skips, plus eight frontend tests. Desktop browser checks cover 74 page/viewport and interaction-state cases. No new distributed replay was needed to build this report.

Acceptance combines conservation equalities, source/aggregate comparisons, recovery digests, the Hadoop independence scan, manifest checks and rejection of altered summary inputs. The meter workspace additionally passes case selection, timestamp/register changes, exception filtering, notes, review state, reset and JSON-export checks at three desktop widths.

The seven skips comprise four Windows rename cases, one inapplicable mutation, one unconstructible duplicate-sheet fixture and one unavailable PySpark check. Skips are excluded from passes.

The bundle contains the report, standalone LaTeX, code snapshot and full dashboard captures. The repository documents setup and a six-minute demonstration using saved results. Raw workbooks and original runtime directories remain separate.

11 Discussion and conclusion

11.1 What the project establishes

The project integrates delivery accounting, bounded streaming, exact reconstruction and independent verification. Across 15 runs, declared source/aggregate comparisons agree; Gold retains the valid event omitted by restrictive Silver, and recovery reproduces four semantic outputs. The dashboard makes these outcomes inspectable alongside source quality and execution cost.

11.2 Evaluation limits

Several limits determine how the results can be used. The workloads are historical replay on one physical machine. Fault schedules are controlled rather than estimated from an Indian utility network. The scale selections are source-identifier prefixes rather than continuous meter-days. Canonicalization is shared by both engines. Absolute timestamps depend on an unconfirmed timezone assumption. Physical measurement accuracy, tariff correctness, live-system reliability and actual financial savings were not evaluated. The final scale evaluation ends at 1M selected source records.

The zero comparable-energy subtotal reflects unchanged retry values, malformed messages accompanying valid originals and unpriced unmatched intervals. Losing an intermediate cumulative reading can preserve endpoint energy. Operational utility adoption therefore requires evaluation on authorized field data with realistic loss histories and continuous analysis windows.

11.3 Future work

Further work could evaluate realistic communication-loss histories, continuous-window consumption effects or cheaper selective verification. Such extensions would require new baselines and experiments. They are outside this submitted scope.

References

[1] M. Li, Z. Wang, Y. Qu, K. M. Chui and M. Leung-Shea, "A multi-year campus-level smart meter database," *Scientific Data*, vol. 11, article 1284, 2024. doi:10.1038/s41597-024-04106-1.

[2] HKUST smart-meter dataset, Dryad, deposited version 5, 1 August 2024. doi:10.5061/dryad.k3j9kd5h6. Registration and package details: repository `docs/dataset-audit.md`.

[3] Brick Schema, "Brick ontology and documentation." `https://brickschema.org/`. Accessed 13 November 2026.

[4] Apache Spark, "Structured Streaming programming guide," 3.5-series documentation. `https://spark.apache.org/docs/3.5.6/structured-streaming-programming-guide.html`. Consulted for watermark and deduplication semantics. Experiment image: Spark 3.5.1.

[5] Apache Kafka, "Design," version 3.7 documentation. `https://kafka.apache.org/37/design/design/`. Accessed 13 November 2026.

[6] Apache Hadoop, "Hadoop Streaming," version 3.4.3 documentation. `https://hadoop.apache.org/docs/r3.4.3/hadoop-streaming/HadoopStreaming.html`. Accessed 13 November 2026.

[7] Maharashtra State Electricity Distribution Co. Ltd., AMISP tender MMD/T-NSC-06/0922, 19 September 2022, section 2.4.4, PDF p. 185. Tender document.

[8] Oracle Utilities Analytics, "Performance," Meter Data Management metric reference. Quality, timeliness and estimation documentation. Accessed 13 November 2026.

[9] E. Garrison and J. New, "Quality control methods for advanced metering infrastructure data," *Smart Cities*, vol. 4, no. 1, pp. 195–203, 2021. doi:10.3390/smartcities4010012.

[10] S. Kumar and A. Subramaniam, "From Meters to Machine Learning: Building AI-Ready Utilities in India," CSIS, 30 October 2026. Original utility survey and analysis.

[11] Comptroller and Auditor General of India, Report No. 2 of 2025, Tripura, year ended 31 March 2024, chapter IV, printed p. 144. Historical smart-grid project observations.